

Module 2 Lab 2 — Textbook Problem 4.2

Multiplication the long way: building `*` out of nothing but addition and loops

Name: Roey Kadur GitHub repo: [**https://github.com/roee0910/cs82a-portfolio/tree/main/module2**](https://github.com/roee0910/cs82a-portfolio/tree/main/module2)

Problem statement (summary)

Write a function that multiplies two whole numbers **without using the `*` operator**, by adding one number to itself repeatedly inside a loop. Do this two ways:

1. Using a `for` loop
2. Using a `while` loop

Then show that both agree with the "real" product computed by Python's built-in `*` operator (used only to *check* the answer, not to compute it).

```
In [1]: def multiply_for(a, b):
        """Multiply two whole numbers using repeated addition inside a for loop.

        Adds `a` to a running total `b` times. No use of the `*` operator.
        """
        result = 0
        for _ in range(b):
            result = result + a
        return result

        # Quick demo: 7 x 6 should be 42
        print("7 x 6 using multiply_for =", multiply_for(7, 6))
        print("Check against built-in *:", 7 * 6)
```

```
7 x 6 using multiply_for = 42
Check against built-in *: 42
```

```
In [2]: def multiply_while(a, b):
        """Multiply two whole numbers using repeated addition inside a while loop.

        A counter is incremented every pass so the loop is guaranteed to stop
        once it reaches b -- otherwise this would be an infinite loop.
        """
        result = 0
        counter = 0
        while counter < b:
            result = result + a
            counter = counter + 1
        return result
```

```
# Quick demo: 7 x 6 should be 42
print("7 x 6 using multiply_while =", multiply_while(7, 6))
print("Check against built-in *:", 7 * 6)
```

7 x 6 using multiply_while = 42
Check against built-in *: 42

Proving it on more than one example

The single 7 x 6 test above could pass by luck, so here both functions are run against several pairs of numbers -- including an edge case where one factor is `0` and another where it's `1` -- and compared against Python's own `*` operator.

```
In [3]: test_pairs = [(7, 6), (9, 4), (12, 0), (5, 1), (8, 8)]

print(f"{'a':>4} {'b':>4} {'a*b (real)':>12} {'for-loop':>10} {'while-loop':>10}")
for a, b in test_pairs:
    real = a * b
    for_result = multiply_for(a, b)
    while_result = multiply_while(a, b)
    match = (real == for_result == while_result)
    print(f"{'a':>4} {'b':>4} {'real':>12} {'for_result':>10} {'while_result':>10} {'match':>10}")
```

a	b	a*b (real)	for-loop	while-loop	match?
7	6	42	42	42	True
9	4	36	36	36	True
12	0	0	0	0	True
5	1	5	5	5	True
8	8	64	64	64	True

How the while loop terminates

The `while` loop in `multiply_while` is controlled by a separate `counter` variable that starts at `0` and is incremented (`counter = counter + 1`) on every pass through the loop body. The loop's condition, `while counter < b`, is re-checked before each iteration, so once `counter` has been incremented `b` times it is no longer less than `b`, the condition becomes `False`, and the loop exits on its own -- it does not depend on `result` or on any external interruption. Because `counter` is guaranteed to increase by exactly `1` each pass and `b` is a fixed whole number, the loop is guaranteed to run exactly `b` times and then stop, which is what keeps it from running forever.

Reproducibility notes

- This notebook was run top to bottom with **Kernel > Restart & Run All** before saving, so every code cell above shows its real output.

- A PDF copy (`module2_lab2.pdf`) was exported immediately after that full run, so the PDF shows the same outputs seen here.
- Repository: [**https://github.com/roee0910/cs82a-portfolio/tree/main/module2**](https://github.com/roee0910/cs82a-portfolio/tree/main/module2)